# 3.5 Making Decisions

Branching on what the user told us

# FOLLOWING A DIFFERENT PATH

```
if (age >= 18) {
    std::println("You are an adult.");
} else {
    std::println("You are a minor.");
}
```

One condition, two possible outcomes.

# COMPARISONS BETWEEN TWO VARIABLES

Comparisons aren't just for one variable against a fixed number - they work between two variables too.

```
if (age == friendAge) {
    std::println("{} and {} are the same age.", name, friendName);
} else if (age < friendAge) {
    std::println("{} is younger than {}.", name, friendName);
} else {
    std::println("{} is older than {}.", name, friendName);
}
```

## THE COMPARISON OPERATORS USED HERE

| OPERATOR | MEANING |
| --- | --- |
| `==` | equal to |
| `<` | less than |
| `>=` | greater than or equal to |

Same operators, applied where they're actually useful - not memorized as an isolated list before you have a reason to care.

3.5 / A GOTCHA

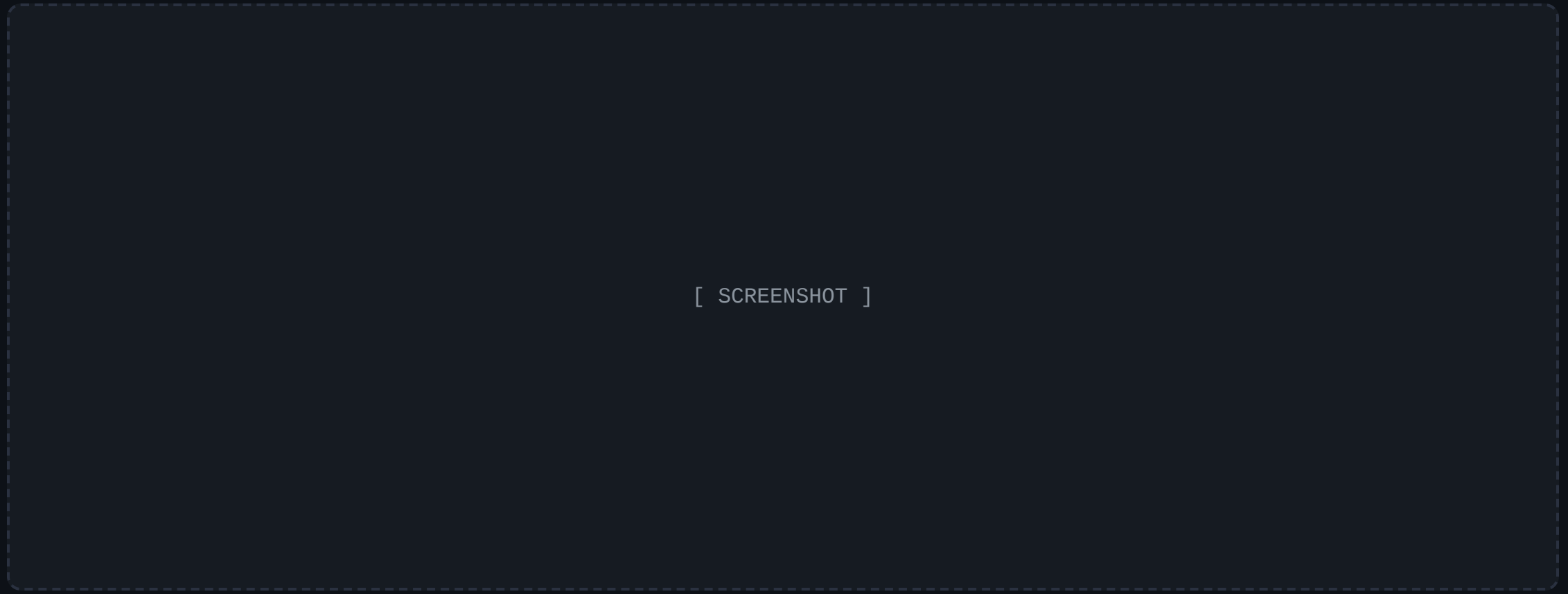
# LEFTOVER NEWLINE

```
std::cin >> age;
std::cin.ignore(); // discard the leftover newline

std::getline(std::cin, friendName); // otherwise this reads an empty line
```

`std::cin >>` leaves the `\n` you pressed sitting in the input buffer - the very next `getline` reads that leftover newline as an empty line if you don't clear it first.

### 3.5 / TRY IT



# ONLY STOP WHEN IT MATTERS

Right-click a breakpoint on the `if` line and add a **condition**, e.g. `age < 18` - the debugger only pauses when that's true, instead of every single run.

[ SCREENSHOT ]

# Next: 3.6

Doing It Many Times